

Working Backwards PRFAQ: CX OS + Coxswain

Chad Hendren

2026-08-09

Contents

Working Backwards PRFAQ	1
CX OS + Coxswain	1
Part 1 - Press Release	2
Headline	2
Subheadline	2
Part 2 - FAQ	3
A. External FAQ (customer-facing)	3
B. Internal FAQ (strategy, tenet conflicts, economics)	6
Part 3 - Leadership one-pager (optional handout)	11
Should we build / continue?	11
Why now?	11
Ask	11
Do not ask yet	11
Document control	11

Working Backwards PRFAQ

CX OS + Coxswain

Customer Experience Operating System powered by a token-frugal coding agent

Field	Value
Document type	Amazon Working Backwards (Press Release + FAQ)
Product name	CX OS (runtime) on Coxswain (cox engine)
Owner	Chad Hendren
Version under review	Coxswain 0.1.0 (private), CXOS fleet green
Date	2026-08-09
Status	Pre-public; architecture and operate loop shipped offline-first
Repos	chendren/coxswain · chendren/CXOS · chendren/TelcoCXOS (all private)
Architecture assets	CXOS-ARCHITECTURE · COXSWAIN-ARCHITECTURE · COMBINED-ARCHITECTURE

Part 1 - Press Release

FOR IMMEDIATE RELEASE

Headline

CX OS on Coxswain Turns a Customer Experience Idea into a Multi-Target, Human-Gated Operating Program in One Session

Subheadline

Domain-agnostic CX programs (retail, financial, healthcare, travel, and more) ship from a closed-world catalog through artifacts, local bind, and plan-only AWS, with an operate loop that proposes work but never silently mutates production.

Seattle / Remote - August 9, 2026 - Today we announce **CX OS**, a Customer Experience Operating System built on **Coxswain**, a local terminal coding agent that treats model selection as a first-class, visible decision.

Most CX “AI” demos invent journeys, invent remediations, and either require a live cloud stack to show anything or write straight into production. CX OS rejects that tradeoff. Teams start from a **closed ontology** of intents, journeys, KPIs, channels, and next-best-action (NBA) rules. One idea string becomes a gated program under `.cox/cx/<name>/`. That program builds **three targets in order**: platform-neutral artifacts, offline local bind, and a **plan-only** AWS CloudFormation package humans apply with scoped credentials. Day-2 operations poll health, simulate traffic, and open **proposals** only. Operators claim work into tasks, close them with audit trails, and hand CAB packages to change boards.

Coxswain is the engine underneath. It amalgamates the best of modern agent CLIs (spec coding and steering docs, agentic tool loops and hooks, plan-first frugality) and adds what none of them make first-class: **three-tier routing** (scout / builder / architect) with **per-call receipts**, an append-only cost ledger, budgets with hard stops, and savings versus an all-flagship baseline. CX domain packs and operate packages live in the same monorepo under a hard **import law**: packages depend on frozen contracts; only the CLI is the composition root.

“We needed something we could demo offline to a compliance team, then hand an AWS plan to a CAB without fear that the tool would CreateStack behind our backs,” said **Maya Chen**, CX Program Lead at a national retail brand (composite scenario from live product walkthroughs). “In one session we stood up a holiday returns surge program, got healthy scores across artifacts, local, and AWS plan targets, and left with an executive brief and CAB export.”

“CX OS is graph-grounded operate, not chatbot theater,” said **Chad Hendren**, principal builder. “Strong nodes decide; weak models generate only where generation is allowed; humans own every mutation. Coxswain keeps the token economics honest so the same team can design, build, and operate without a blank-check model bill.”

Availability. Coxswain and the domain-agnostic CXOS workspace ship today for private design partners and internal workshops. Requires Node 20+ and pnpm. Offline commands work without API keys. Live and hybrid modes light up when local platforms or model keys are ready. Public packaging and `v0.1.0` tagging follow live-API smoke completion.

Hard rules customers can quote:

1. No silent production mutation
2. AWS is plan-only from Coxswain
3. Offline-first
4. Strong graph first

5. Import law (CLI sole composition root)

Learn more: private repositories chendren/coxswain and chendren/CXOS; architecture diagrams in this pack; `pnpm cox cx doctor` and `pnpm cox cx board` for the first five minutes of truth.

Part 2 - FAQ

A. External FAQ (customer-facing)

1. What is CX OS?

CX OS is a **Customer Experience Operating System**: a closed-world build-and-operate system for CX programs. It turns a natural-language idea into multi-target design artifacts and an operate loop (health -> proposal -> task -> done), grounded in ontology packs rather than open-world agent improvisation.

2. What is Coxswain?

Coxswain (cox) is a **local, terminal-native coding agent** that:

- Plans with **spec coding** (requirements -> design -> tasks with human approval gates)
- Keeps project truth in **steering docs** (plus CLAUDE.md / AGENTS.md import)
- Runs an **agentic tool loop** with permission modes and lifecycle hooks
- **Routes every model call** to scout, builder, or architect, and **prints the receipts**

CX OS is the CX domain and operate surface on top of Coxswain (`pnpm cox cx ...`).

3. Who is this for?

Primary jobs-to-be-done:

Persona	Primary value
CX Product Manager	Gated programs, shared language, explainable NBA
Contact Center / CX Solutions Architect	Multi-target design + plan-only AWS handoff
Journey Owner / Ops Lead	Human-gated day-2 proposals and tasks
GenAI / Graph Engineer	Strong/weak boundary, path audits
Change / Security / Compliance	No silent prod write; audit + APPLY.md
AWS PS / Partners	Offline golden demos + customer-scoped workspaces
Workshop facilitators	Teachable offline-first graph-node practice

4. What problem does this solve that existing tools do not?

Three failure modes dominate CX AI tooling:

Failure	What buyers feel	CX OS counter
Open-world drift	Invented journeys and remediations	Closed ontology packs; pure NBA match
Unsafe automation	Fear of bots rewriting Connect/prod	Plan-only AWS; console proposes only

Failure	What buyers feel	CX OS counter
Demo friction	Needs cloud + keys to show anything	Offline-first adapters and golden path

Coding agents alone solve implementation; contact-center suites alone solve runtime. CX OS + Coxswain close the loop from **idea** -> **design** -> **multi-target build** -> **gated operate** -> **CAB handoff** in one workspace.

5. How does a first successful session look?

Domain-agnostic workspace (~ /CXOS):

```
export COXSWAIN_ROOT=~ /coxswain
cd ~ /CXOS
pnpm cox cx doctor
pnpm cox cx run core "Customer experience for a national retail brand: returns, loyalty, store pi
pnpm cox cx board
pnpm cox cx brief core
pnpm cox cx dashboard ./cxos-dash-board.html
pnpm cox cx cab-export core
```

Observed product path (retail holiday scenario): create -> approve requirements -> build artifacts/local/aws offline -> healthy status -> simulate -> report -> next steps for console, apply, board, brief, CAB.

6. Which industries are supported today?

Domain-agnostic primary via registry detection and packs:

Pack / program	Vertical	Example journeys
core + retail pack	Retail	Returns, loyalty, pickup, order support, retention
fin-core	Financial	Account inquiry, fraud, loan, onboarding, retention
health-core	Healthcare	Appointments, claims, prior auth, benefits
travel-core	Travel	Booking, disruption, loyalty, check-in, retention
holiday-returns-2026	Retail surge	Dec-Jan returns volume + loyalty + CAB
default ontology	Cross-industry seed	Billing, support, account, sales, compliance, public sector domains
TelcoCXOS (separate repo)	Telco demo	Triggers on telco/mobile/broadband keywords

Registry scoring routes idea strings to packs (e.g. retail ~0.85, telco ~0.12 for non-telco retail language).

7. Does this deploy AWS for me?

No. Coxswain writes **plan-only** template.yaml and APPLY.md. A human applies with scoped credentials. Coxswain never runs CreateStack. That is a product promise, not a temporary limitation.

8. Will the operate daemon change production without me?

No. Console, watch, and daemon **propose only**. apply / claim creates a **task + remediation note**. Operators execute remediations outside silent auto-mutation. Every control path returns a path[] audit trail.

9. Can I use it offline / air-gapped?

Yes. Default runtime is offline when live flags and keys are absent. Artifacts, local, and AWS adapters have offline implementations under `.cox/cx/`. Tests and workshops are designed to pass without network. Live/hybrid modes prefer real platform/models when healthy.

10. How does model cost work?

Coxswain routes by tier and task complexity:

Tier	Role	Typical use
scout	Cheap classify / explain / mechanical	Hooks, classification, simple edits
builder	Routine implementation	Spec tasks, tests
architect	Requirements, design, review	Escalation target

Ledger records every call (including router classification), cache reads, and **savings vs all-architect baseline**. Budgets warn and can degrade tiers; hard-stop at 100% until extended. Local Ollama models can price at \$0 for workshop economics.

11. How is this different from Claude Code, Kiro, Copilot CLI, or Grok Build alone?

Coxswain **borrow**s strengths (spec/steering, agent loop/hooks, explain/suggest, plan-first frugality) and **own**s visible multi-tier routing + ledger + CX OS domain operate. Those CLIs do not ship a closed-world multi-target CX program with human-gated operate and plan-only AWS CAB export as a first-class product surface.

12. What is the “seven-layer OS” people show on the architecture diagram?

Layer	Responsibility
01 Catalog	Ontology packs, strong graph, journeys, intents/KPIs/NBA, channels
02 Program	Spec lifecycle, multi-target build, design merge, approval gates
03 Observe	Health, doctor, simulate, report (read-only)
04 Operate	Console tick, proposals, tasks, watch/daemon (propose-only)
05 Fleet	Board, queue, dashboard, fleet-status, board-sync
06 Govern	Brief, audit, snapshot, CAB export, export-AWS
07 Fabric	SQLite, ledger cache, logger/OTel, healthz, Docker/CI

13. What does “engine x fleet” mean?

Coxswain (engine) is the build-time / runtime monorepo (~26 packages: coding agent + CX domain packages + packs).

CXOS (fleet) is the workspace of programs (specs) and operate surfaces. Combined flow:

idea -> pack detect -> seed -> plan -> build (artifacts -> local -> aws) -> health -> propose -> task -> close -> brief/CAB

14. Is source available? Is it open source?

Today repositories are **private** on GitHub (chendren/coxswain, chendren/CX0S, chendren/TelcoCX0S). No public releases yet. Open-source decision is deferred (see Internal FAQ).

15. What do I need installed?

- Node ≥ 20 , pnpm
- Clone coxswain, pnpm install
- Clone CXOS (no install required for normal use; pnpm cox proxies into COXSWAIN_ROOT)
- Optional: Ollama + local models, local omnichannel platform for live local bind
- Optional: Anthropic / xAI keys for live model tiers

16. How do I trust what the system did?

- Spec phases with explicit approve gates
 - Append-only audit.jsonl and deploy history
 - path[] on ops surfaces
 - Ledger JSONL for model economics
 - CAB package: MANIFEST, BRIEF, plan-only CFN, remediations
 - Health history samples from status
-

B. Internal FAQ (strategy, tenet conflicts, economics)

1. Why are we building this (tenet: Customer Obsession)?

Customers (CX PMs, SAs, ops, compliance, PS) are stuck between slideware, unsafe bots, and demos that die without cloud. The product they hire is **trustworthy speed**: closed catalog + multi-target artifacts + gated operate + AWS handoff they can defend in a CAB. We work backwards from that job, not from “another coding agent.”

2. What is the one-sentence product definition we refuse to dilute?

Closed-world, offline-first, human-gated CX build-and-operate on a token-frugal local agent, with plan-only AWS.

If a feature invents open-world remediations, auto-mutates prod, or requires live AWS to demo core value, it is out of bounds for v1.

3. What is the customer narrative we sell in the first 10 minutes?

1. Doctor + board show a healthy fleet.
2. cx run from an idea string produces healthy multi-target deployments offline.
3. Brief + dashboard + CAB export prove governability.
4. Console produces proposals only; claim/apply makes human-owned tasks.
5. Ledger (when models used) shows tier mix and savings.

4. Why two products (Coxswain + CX OS) instead of one brand?

Layer	Why separate
Coxswain	General coding agent value: routing, specs, steering, hooks, ledger
CX OS	Domain product: ontology, packs, multi-target CX, operate, fleet, CAB

Composition is intentional: CLI wires both; import law keeps CX packages from becoming a ball of mud. TelcoCXOS remains a **vertical demo workspace**, not the primary domain-agnostic product (CX0S).

5. What does GitHub status tell us about readiness (as of 2026-08-09)?

Repo	Visibility	Open issues / PRs	Notes
chendren/coxswain	Private	0 / 0	TypeScript monorepo; CI, Release, Scorecard workflows active; private: true, version 0.1.0; main pushed 2026-08-09; local main ahead of origin by 2 commits (travel pack + test stabilize)
chendren/CX0S	Private	0 / 0	Domain-agnostic workspace; created 2026-08-09; local main ahead by 2 commits (holiday returns + travel-core); architecture diagrams untracked locally
chendren/TelcoCX0S	Private	-	Telco demo companion; pushed 2026-08-09

Interpretation: Product surface is advanced for a private 0.1.x; process maturity (issues, public PRs, tagged release) has not started. No public release artifacts. Security policy and SBOM/scorecard path exist for hardening.

Engine maturity signals (docs + tree):

- ~26 packages in pnpm workspace (coding agent + CX domain + 4 vertical packs + registry)
- Offline e2e and package tests; README still cites ~895 tests for core agent M2 story (CX expansion added more)
- Stages shipped: pack registry, retail/financial/healthcare/travel packs, SQLite proposals/tasks, cx-ops facades (journey/knowledge/agent/analytics/govern), fleet board/queue/dashboard, CAB/brief/audit/snapshot
- **Remaining for advertised M3 / public 0.1.0:** live-API smoke for pure coding agent path; push unpushed commits; optional architecture asset commit

6. What are we not doing in v1 (explicit non-goals)?

- Auto CreateStack / live Connect mutation from Coxswain
- Open-world freeform NBA without ontology match
- Windows-first packaging
- Full MCP server marketplace (registry designed to allow later)
- Subagent 8-way fan-out as a product guarantee (event model allows later)
- Claiming live multi-tenant SaaS CX platform status (this is local-first OS + plan export)

7. How do we measure success (input / output metrics)?

North-star customer outcomes

Metric	Definition	v1 target (design partner)
Time-to-healthy program	Idea -> multi-target healthy offline	< 15 minutes for trained user
Gate compliance	% prod mutations with human claim/apply	100% (by architecture)
CAB readiness	Specs that export brief + plan CFN without rework	≥ 80% of partner runs
Offline demo success	Golden path without keys	100% in CI
Token savings	Ledger savings vs all-architect baseline	≥ 50% on mixed sessions (stretch 70%+ per engine claims)

Product health

Metric	Definition
Fleet score	Board: specs healthy / total
Proposal hygiene	Open proposals age and claim latency
Path audit coverage	Ops commands emitting path []
Typecheck / test green	CI on main

8. What is the business model (working assumption)?

Not locked. Candidate models for later:

1. **Open core engine** + paid vertical packs / PS enablement
2. **Private partner license** for AWS PS and SI workshops
3. **Workshop + implementation** services wrapped around the OS

Near-term: maximize design-partner learning and CAB-safe demos. Do not optimize pricing before M3 live smoke and two external partner runs.

9. Competitive alternatives and why we still win

Alternative	Gap vs CX OS + Coxswain
Generic coding agents	No closed CX catalog, multi-target CX deploy, gated operate, CAB
Contact center admin consoles	No idea->program offline design loop; heavy cloud
Pure GraphRAG demos	Rarely human-gated ops + plan-only AWS + ledger economics
Homegrown scripts	No import law, tests, fleet board, audit path

Our unfair combination is **strong graph + frugal agent + plan-only cloud + offline-first operate**, enforced as hard rules in code and docs.

10. What are the top risks and mitigations?

Risk	Mitigation
Ontology packs lag real customer domains	Pack registry + vertical packs; absorb weak only into strong hubs
Users expect auto-remediation of prod	Product language, UX copy, APPLY.md; refuse CreateStack
Token cost still high on architect-heavy design	Routing policy, budgets, Ollama workshop profiles
Monorepo complexity / God modules	Staged splits (cx-journey, cx-knowledge, facades); import law
Private-only limits feedback	Design-partner program; keep issues internal until readiness
Security of bash tool / agent host	Permission modes, deny lists, SECURITY.md threat model, no silent prod

11. Architecture tenets (non-negotiable)

1. **No silent prod mutation**
2. **AWS is plan-only**
3. **Offline-first**
4. **Strong graph first** (weak models optional, constrained)
5. **Import law** (cx-* -> only @cox/core + @cox/cx-core; CLI sole composition root)
6. **Receipts on screen** (routing, ledger, path audits)
7. **Human gates on phases** (requirements/design/tasks)

12. Tenet conflicts we already resolved

Conflict	Decision
Speed vs safety	Offline speed + propose-only ops; never auto CreateStack
Domain depth vs generality	Domain-agnostic CXOS primary; TelcoCXOS as keyword-triggered demo
One mega package vs many	Prefer package splits with frozen contracts; CLI wires

Conflict	Decision
LLM creativity vs catalog truth	Weak generate allowed; NBA/console routing pure graph

13. What is the launch bar (Working Backwards exit criteria)?

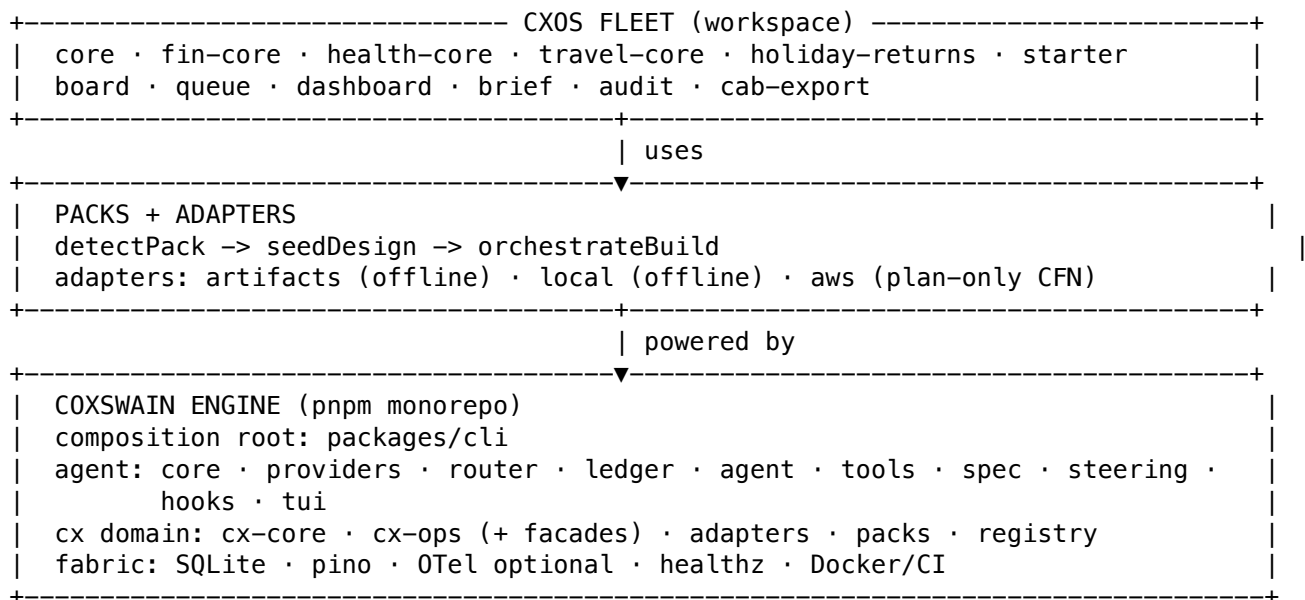
Press-release true when:

1. Unpushed main commits for coxswain + CXOS are pushed or intentionally held with written reason
2. Live-API smoke (M3) for coding-agent golden path documented green
3. Offline CX golden path green in CI on every main commit
4. Architecture pack (three diagrams + this PRFAQ) committed to a durable location
5. At least one external design partner completes holiday-or-vertical scenario and CAB export without prod mutation
6. SECURITY.md + CI scorecard remain green; no known critical unfixed host-escape in default permission mode

14. Dependencies outside our control

- Model provider APIs and pricing
- Optional local omnichannel platform readiness (CX_PLATFORM_DIR)
- Customer AWS account permissions for human CFN apply
- Ollama / local GPU availability for zero-dollar workshops

15. Appendix: system context map (one page)



16. Appendix: illustrative end-to-end flow (retail holiday)

From product scenario (Maya Chen, 2026-08-08):

1. cx board -> multi-spec fleet (retail, fin, health, starter)
2. cx catalog -> closed domains, KPIs, NBA rules
3. cx run holiday-returns-2026 "..." -> offline wiring; 6 artifacts; local + aws healthy; simulate scores
4. Next: console -> claim/apply -> tasks -> brief -> cab-export -> daemon (optional)

This is the press-release moment: **healthy multi-target program + govern package without live AWS mutation.**

Part 3 - Leadership one-pager (optional handout)

Should we build / continue?

Yes, as a private design-partner product, if we hold the hard rules and finish M3 live smoke before any public claim of 0.1.0 GA.

Why now?

Coding agents are commodity; **trusted CX program operate** is not. Offline + plan-only + closed world is a differentiated story for PS, workshops, and regulated LOBs.

Ask

1. Protect the five hard rules as product law.
2. Fund design-partner runs (retail + one regulated vertical).
3. Sequence: push hygiene -> M3 smoke -> partner CAB demo -> packaging decision (private license vs open core).

Do not ask yet

Public launch marketing, multi-tenant SaaS CXOS, or automatic production remediation.

Document control

Version	Date	Change
0.1	2026-08-09	Initial Working Backwards PRFAQ from architecture diagrams, CXOS/Coxswain docs, and GitHub private repo status

Sources consulted

- Architecture: CX0S-ARCHITECTURE, COXSWAIN-ARCHITECTURE, COMBINED-ARCHITECTURE (Documents pack + CX0S/docs, coxswain/docs)
- Product docs: docs/00-OVERVIEW.md, 01-ARCHITECTURE.md, CX0S.md, CX0S-COMPLETE.md, CX0S-PERSONAS-USE-CASES.md, wave/complete/superheavy summaries, USER-SCENARIO0-2026-08-08.md
- Repos: local trees + gh metadata for chendren/coxswain, chendren/CX0S, chendren/TelcoCX0S

Related diagrams (this folder)

- CX0S-ARCHITECTURE.svg / .png
- COXSWAIN-ARCHITECTURE.svg / .png
- COMBINED-ARCHITECTURE.svg / .png